

AI WEATHERWISE API – Intelligent Weather Forecasting & AI Recommendation Backend

Node.js, Express.js, MongoDB, Gemini AI, Thunder Client, Open Weather

Team Members:

- **Sairunisha S M** – Team Leader
- **Swetha R**
- **Lokeshwari P**
- **Yuvashri R**

1. Project Overview

AI WeatherWise API is a secure RESTful backend application developed using Node.js, Express.js, MongoDB, and Google Gemini AI. The system provides real-time weather information and AI-generated weather insights through a scalable API architecture.

The platform enables users to register, log in securely, save favorite locations, retrieve live weather conditions, and receive intelligent recommendations based on environmental conditions.

Technologies Used

- Backend: Node.js, Express.js
- Database: MongoDB with Mongoose
- Authentication: JWT (JSON Web Token)
- Password Security: bcryptjs
- AI Integration: Google Gemini AI
- API Testing: Thunder Client / Postman

Objective

To develop a secure and scalable backend weather intelligence platform that combines real-time weather information with Artificial Intelligence to provide personalized weather insights and recommendations.

2. Features

Authentication Features

1. User Registration
2. User Login
3. JWT Authentication
4. Password Encryption using bcrypt

Weather Features

5. Fetch Real-Time Weather Data
6. Generate AI Weather Summary
7. Generate Personalized Weather Recommendations

Location Management Features

8. Save Favorite Locations
9. View Saved Locations
10. Update Favorite Locations
11. Delete Favorite Locations

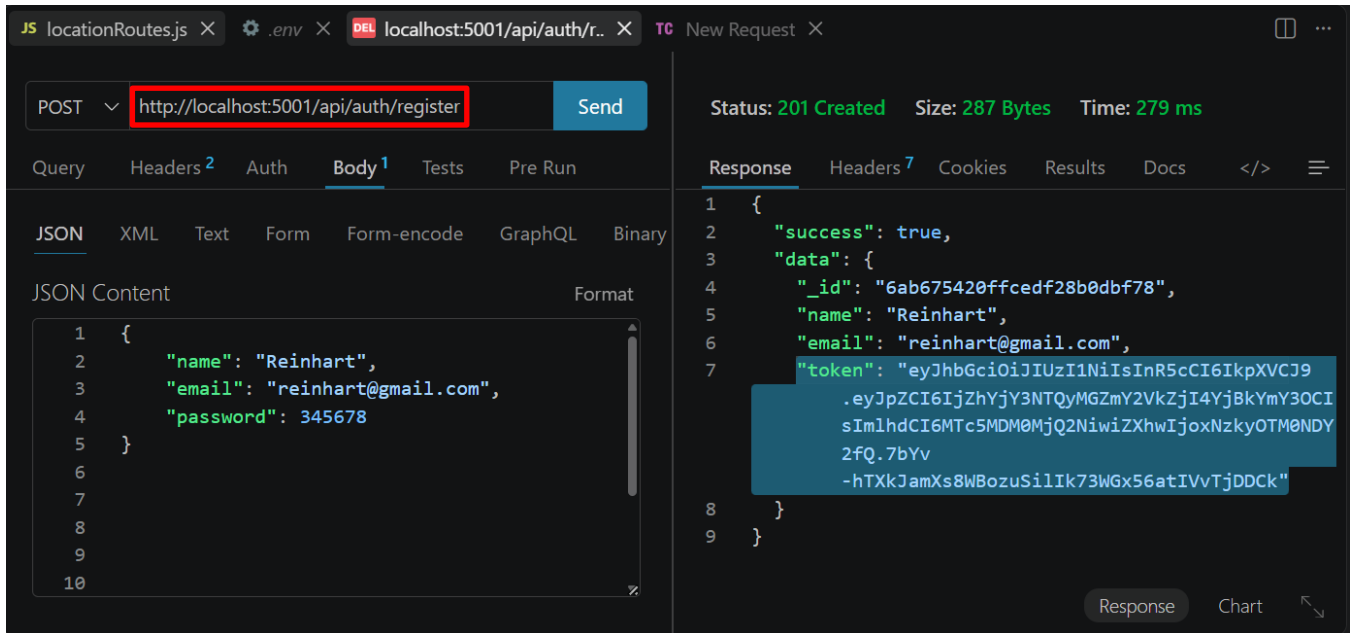
3. Project Architecture

Folder Structure



4. API Endpoints & Thunder Client Screenshots

I. User Registration



Register endpoint allows new users to create an account in the AI WeatherWise system.

The user provides:

- Name
- Email
- Password

The password is encrypted using bcrypt before being stored in MongoDB.

Upon successful registration:

- User account is created
 - JWT token is generated
 - User information is returned
-
- success indicates successful registration.
 - token is used for future authenticated requests.
 - Password is securely hashed and never stored as plain text.

II. User Login

The screenshot shows a REST client interface with a POST request to `http://localhost:5001/api/auth/login`. The request body is a JSON object containing email and password. The response is a 200 OK status with a JSON body containing success, data (with id, name, email, and token), and headers.

```
POST http://localhost:5001/api/auth/login
```

```
{
  "email": "reinhart@gmail.com",
  "password": "345678"
}
```

Status: 200 OK Size: 287 Bytes Time: 134 ms

```
{
  "success": true,
  "data": {
    "_id": "6ab675420ffcedf28b0dbf78",
    "name": "Reinhart",
    "email": "reinhart@gmail.com",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhYjY3NTQyMGZmY2VkZjI4YjBkYmY3OCIsIm1hdCI6MTc5MDM0MjY5NCwiZXhwIjozOTM0Njk0fQ.HRyzFY7KJdXLhIrX3apAAF1yMnewRhfccthqVAX-wGsg"
  }
}
```

Allows registered users to log into the system.

The endpoint validates:

- Email
- Password

If credentials are correct, a JWT token is returned.

Without this JWT token, users cannot access protected weather or location management route.

III. View Profile

The screenshot shows a REST client interface with a GET request to `http://localhost:5001/api/auth/profile`. The response is a 200 OK status with a JSON body containing success, data (with id, name, email, and createdAt), and headers.

```
GET http://localhost:5001/api/auth/profile
```

```
{
  "city": "Haryana",
  "temperature": 31.56,
  "humidity": 40,
  "condition": "Clear"
}
```

Status: 200 OK Size: 144 Bytes Time: 27.78 s

```
{
  "success": true,
  "data": {
    "_id": "6ab675420ffcedf28b0dbf78",
    "name": "Reinhart",
    "email": "reinhart@gmail.com",
    "createdAt": "2026-09-25T13:21:06.362Z"
  }
}
```

IV. Save Favorite Location

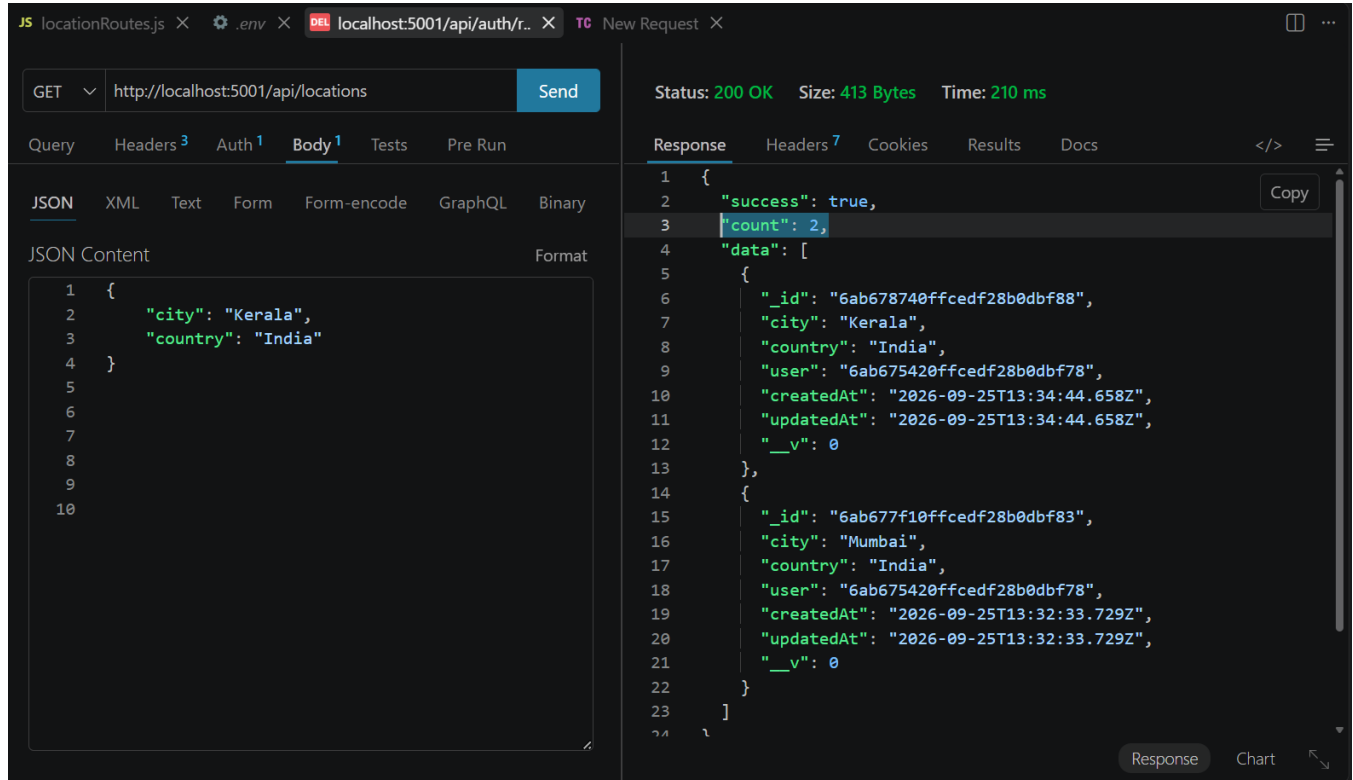
The screenshot shows a REST client interface with a POST request to `http://localhost:5001/api/locations`. The request body is a JSON object: `{ "city": "Mumbai", "country": "India" }`. The response status is `201 Created` with a size of `212 Bytes` and a time of `292 ms`. The response body is a JSON object: `{ "success": true, "data": { "city": "Mumbai", "country": "India", "user": "6ab675420ffcedf28b0dbf78", "_id": "6ab677f10ffcedf28b0dbf83", "createdAt": "2026-09-25T13:32:33.729Z", "updatedAt": "2026-09-25T13:32:33.729Z", "__v": 0 } }`.

The screenshot shows a REST client interface with a POST request to `http://localhost:5001/api/locations`. The request body is a JSON object: `{ "city": "Kerala", "country": "India" }`. The response status is `201 Created` with a size of `212 Bytes` and a time of `233 ms`. The response body is a JSON object: `{ "success": true, "data": { "city": "Kerala", "country": "India", "user": "6ab675420ffcedf28b0dbf78", "_id": "6ab678740ffcedf28b0dbf88", "createdAt": "2026-09-25T13:34:44.658Z", "updatedAt": "2026-09-25T13:34:44.658Z", "__v": 0 } }`.

Allows an authenticated user to save a favorite city.

Each saved location is associated with the authenticated user through their unique user ID.

V. Get All Favorite Location



GET `http://localhost:5001/api/locations` Send

Status: 200 OK Size: 413 Bytes Time: 210 ms

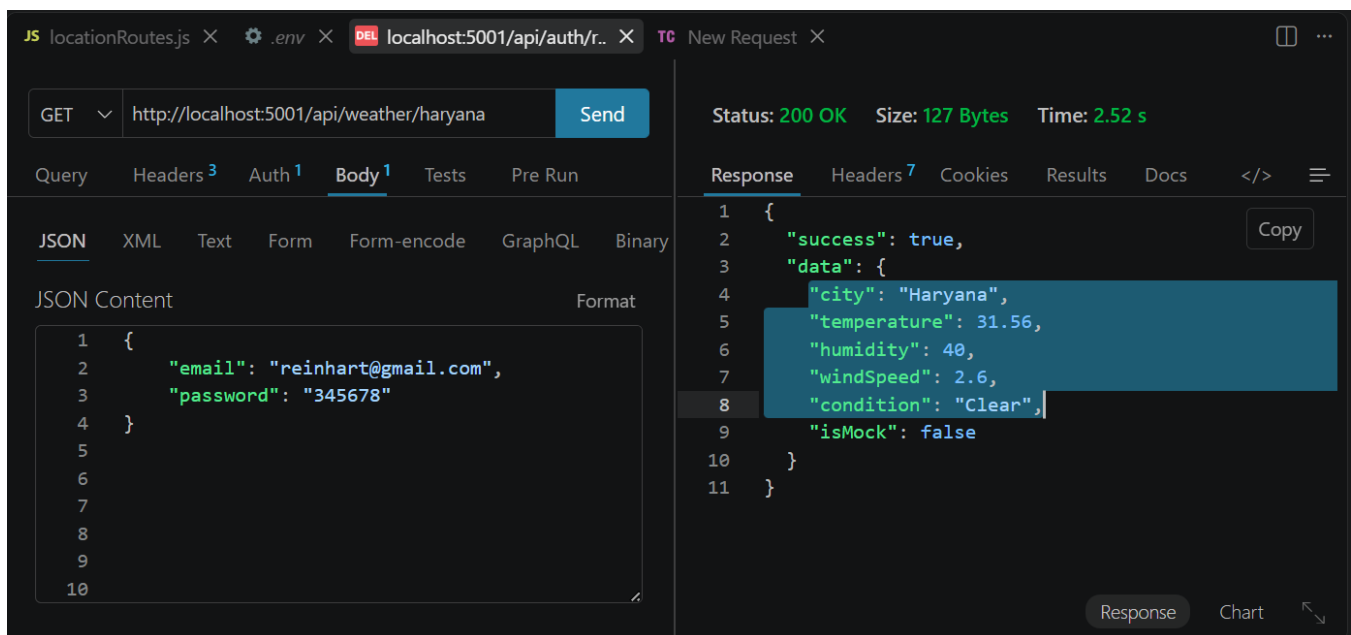
Response Headers 7 Cookies Results Docs

```
1 {
2   "success": true,
3   "count": 2,
4   "data": [
5     {
6       "_id": "6ab678740ffcedf28b0dbf88",
7       "city": "Kerala",
8       "country": "India",
9       "user": "6ab675420ffcedf28b0dbf78",
10      "createdAt": "2026-09-25T13:34:44.658Z",
11      "updatedAt": "2026-09-25T13:34:44.658Z",
12      "__v": 0
13    },
14    {
15      "_id": "6ab677f10ffcedf28b0dbf83",
16      "city": "Mumbai",
17      "country": "India",
18      "user": "6ab675420ffcedf28b0dbf78",
19      "createdAt": "2026-09-25T13:32:33.729Z",
20      "updatedAt": "2026-09-25T13:32:33.729Z",
21      "__v": 0
22    }
23  ]
24 }
```

This endpoint retrieves all favorite locations saved by the logged-in user.

This feature allows users to maintain and access multiple favorite locations from a single account.

VI. Fetch Weather Data



GET `http://localhost:5001/api/weather/haryana` Send

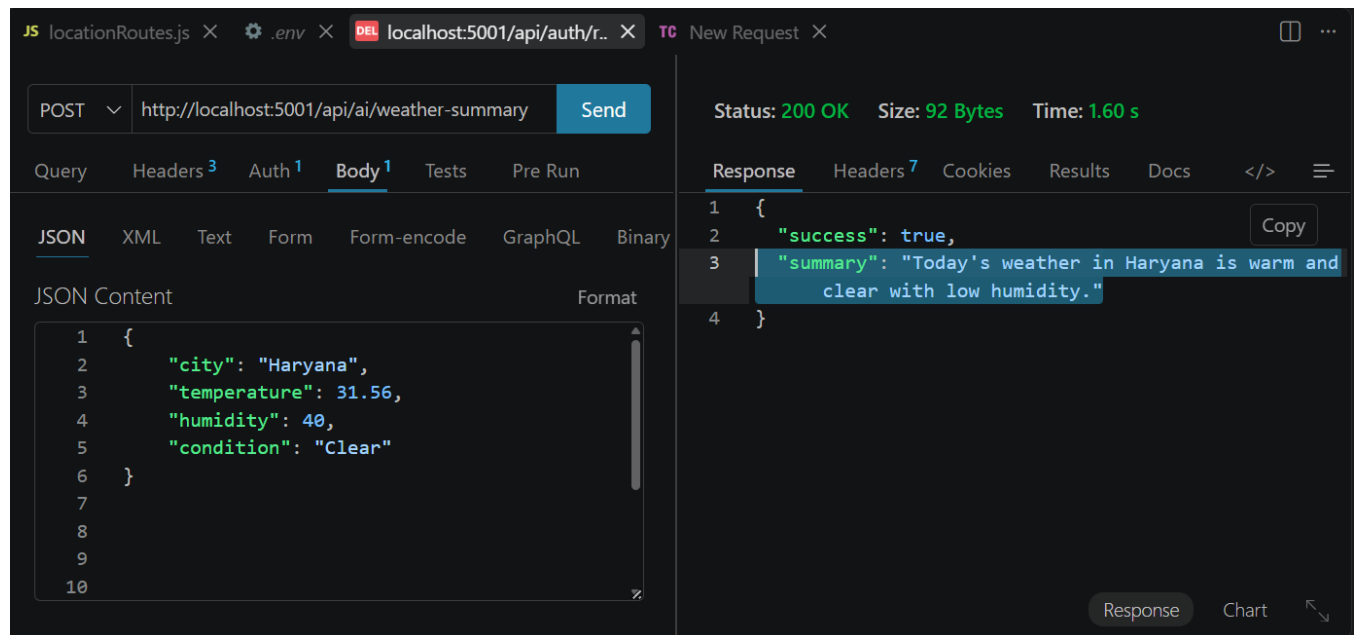
Status: 200 OK Size: 127 Bytes Time: 2.52 s

Response Headers 7 Cookies Results Docs

```
1 {
2   "success": true,
3   "data": {
4     "city": "Haryana",
5     "temperature": 31.56,
6     "humidity": 40,
7     "windSpeed": 2.6,
8     "condition": "Clear",
9     "isMock": false
10  }
11 }
```

Retrieves real-time weather information from the integrated weather API.
The endpoint provides current weather metrics that serve as the input for Gemini AI recommendations.

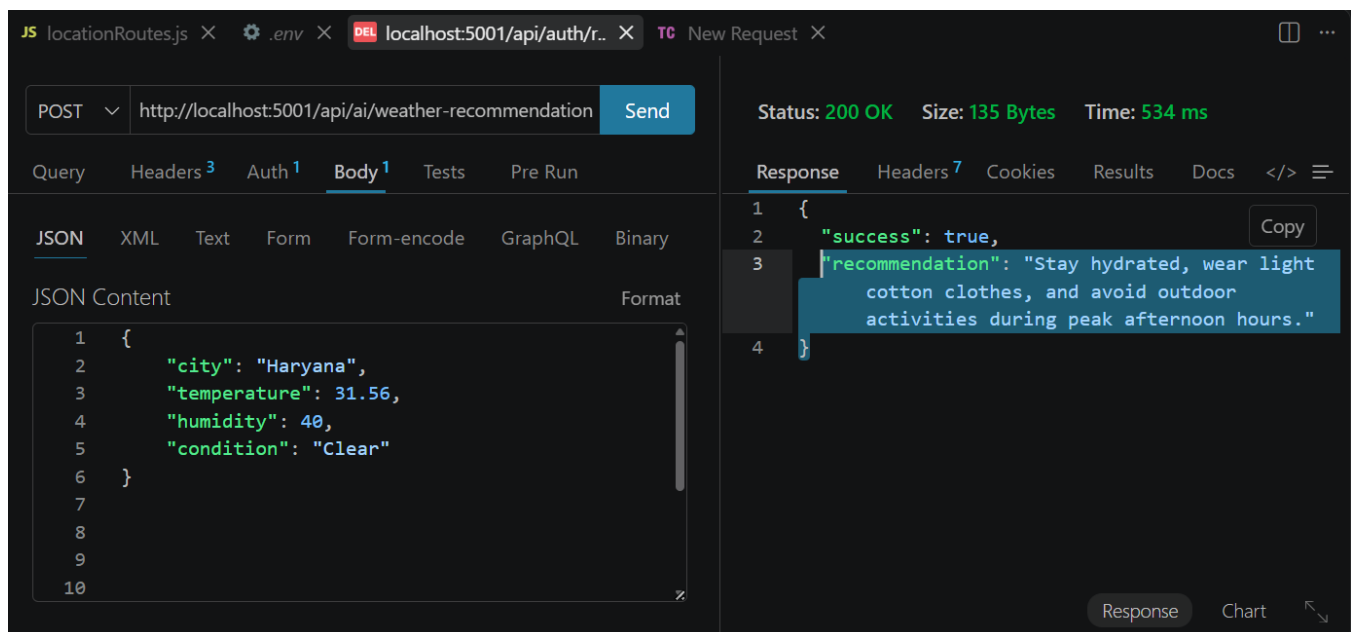
VII. Generate AI Weather Summary



Weather-summary endpoint sends weather metrics to Google Gemini AI and generates a natural-language weather summary.

Instead of presenting only raw weather data, Gemini AI converts complex weather metrics into user-friendly explanations.

VIII. Generate AI Recommendations

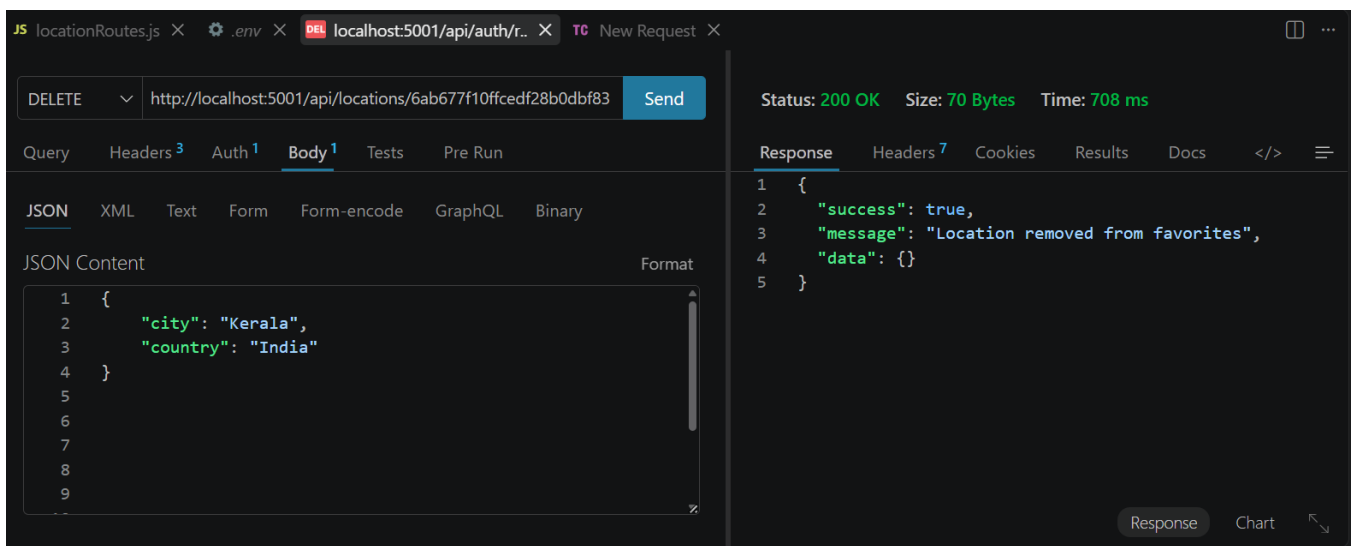


Provides personalized suggestions based on weather conditions.

This feature enhances decision-making by suggesting:

- Suitable clothing
- Outdoor activities
- Travel recommendations
- Safety precautions

IX. Delete Favorite Location



5. Conclusion

The AI WeatherWise API successfully combines modern backend technologies with Artificial Intelligence to deliver intelligent weather forecasting services. Through secure authentication, location management, real-time weather retrieval, and Gemini AI integration, the system provides users with meaningful weather insights rather than simple numerical data.

The project demonstrates the practical implementation of RESTful API development, MongoDB database management, JWT authentication, and AI-powered recommendation systems, making it a scalable and industry-relevant weather intelligence platform.

Demo link: <https://drive.google.com/file/d/12ex-fTwDsUWwNQIHs8plqel3ugn5DMm2/view?usp=drivesdk>

GitHub link: <https://github.com/sairunisha-sm/AI-WeatherWiseAPI.git>